

VECTA — 進捗レポート 2026-07-26

概要

仕組み

検証

リスクと残タスク

用語集

検証

Abstract

What lint・型・テスト・ビルド・バンドル走査・漏洩監査・独立レビュー・CI を通した。

Why 触ったのはアクセス拒否の中核。動くことだけでなく、拒否が壊れていないことを示す必要がある。

How 検査には「必ずヒットする対照」を入れ、0件が「安全」なのか「検査できていない」のかを区別した。

lint / typecheck / build

緑

ユニット・結合テスト 435 件

緑

クライアントバンドル走査

0/22

push 前 漏洩監査

0件

Fable 独立レビュー

承認

GitHub Actions CI

success

未確認

本番の実レイテンシ（未デプロイのため）
往復回数は追跡と計数、時間は見積り

この Mac でのみ失敗

persistence 結合テスト（並列時タイムアウト）
直列なら 46/46 緑・CI も緑

実施しなかった

スクリーンショット（画面に変化がないため）
アプリバーは実コンポーネントのテストで担保

通った検査と、通していない・通らなかった検査。右側を落とさないのがこのページの目的。

テスト件数の内訳

- domain 32 / application 70 / persistence 46 / **web 268**（変更前 264、**+4**） / operations 19
- 増えた4件はいずれもこの変更の主張そのものを固定するもの（次節）

Detail

1. 追加したテスト4件

テスト	何を固定するか
membership は DB 往復ゼロで読める	書き込みパスの往復削減の前提。ワークスペース読み取りが 0 回であることを数える
解決済み project 行とビューが1回の読み取りを共有する	今回の主張そのもの。並列に呼んでも読み取りは 1 回
読み取りは membership のテナントに限定される	ID 単独で行に届かないこと。(tenantId, projectId) で呼ばれることを確認
行が消えていても同一の不透明な 404	ステータスだけでなく本文 null まで一致することを確認

2. 実測と見積りの区別

実測したもの: テスト件数、シームでの呼び出し回数 (1回)、バンドル走査のヒット数、CI の成否。

コードから追ったもの: 往復が3本から2本になること。principal 読み取りが db.batch 1本、ワークスペース読み取りが db.batch 1本であることを実装で確認したうえでの結論。

見積りにすぎないもの: 「約70ms 短縮」。既存の計測値である「東京⇄シンガポール 1往復 約70ms」に、削れた往復数を掛けた数字。**本番での前後比較は行っていない** (デプロイが CI 経由+人間承認のため、まだ本番に出ていない)。

3. クライアントバンドル走査

今回の変更で、ミドルウェアが @vecta/persistence を直接 import するようになりました。ブラウザに配られる成果物にサーバー専用のものが混ざっていないかを、ビルド後の build/client に対して確認しています。

走査対象	パターン	ヒット
build/client の 29 アセット	秘密の名前 (DATABASE_URL ほか)、drizzle / neondatabase / node-postgres / jose、今回触ったサーバー側識別子 — 計 22	0

ただしこれは**手作業の走査**で、自動では回っていません。この「構造で強制する」仕組みづくりが、HANDOFF の次の作業項目です。

4. しくじった検査 — 監査 grep を3回書き直した

push 前の漏洩監査で、最初にした検査は**全パターン 0 ヒットで「クリーン」**に見えていました。実際には何も検査できていませんでした。原因は2つ重なっていました。

- この環境の grep は **ugrep** の実体で、複数ファイル指定や -U の解釈が GNU/BSD grep と異なる
- zsh は変数展開を**単語分割しない**。FILES=\$(...) を grep ... \$FILES に渡すと 12 個のファイルが**1個の引数**になり、ファイルが見つからないまま 0 ヒットが返る

さらに NUL バイト検査は `$('#x00')` が zsh で空文字列になるため、**全ファイルが陽性**という逆方向の誤りも出しました。

直し方は、実体の `/usr/bin/grep` を明示し、ファイルパスをコマンドラインに直書きし、**必ずヒットする対照パターンを監査に同梱**することです。対照が 0 件を返したら、その監査は壊れている——この判別ができない検査は、合格しても何も証明しません。最終結果は対照付きで取り直したもので、ヒットは HANDOFF に元からある公開リポジトリパス1件のみです。

5. この Mac でのみ落ちたテスト

`packages/persistence` の結合テストが、フルの `pnpm check` で2回とも同じ2件失敗しました。原因は環境です。

条件	結果
8ファイル並列（既定） — それぞれが Postgres コンテナを起動	2件失敗（5秒のテストタイムアウト超過 → 以降 <code>current transaction is aborted</code> で連鎖）
<code>--fileParallelism=false</code> （直列）	46/46 緑
GitHub Actions CI (ubuntu-latest)	success （今回のコミットを含め直近9ラン連続）

セットアップ側 `beforeAll` には60秒の猶予がある一方、テスト本体は vitest 既定の5秒のままです。コードの欠陥ではなく環境の制約と判断し、発火条件を付けて債務に記録しました（[リスクと残タスク](#)）。ただしローカルの `pnpm check` が一発で緑にならない状態であることは事実です。

6. 独立レビュー

設計レビューを Fable に依頼しました（実装は書かせず、助言のみ）。往復数の主張、セキュリティ3性質、context の設定漏れ経路、レイアウト削減の妥当性、読み取り重量の回帰、記憶つき読み取りの失敗キャッシュ挙動を対象にしています。

- 往復 3→2 の主張は実コードで裏付けられた。増えた経路はなし
- セキュリティ3性質は維持。新しい存在オラクルはなし
- ただし書き込みパスの応答本文に差分があると指摘。既存形状の発生窓が広がったもの（[詳細](#)）
- ダッシュボードの読み取り重量は債務記録が妥当、と同意

指摘のうち事実関係（`prefetch="intent"` の位置、クライアント側の `NOT_FOUND` 型契約）は、こちらでもコードを開いて確認しました。

1. Vitest — Test timeout / file parallelism — vitest.dev/config
2. Testcontainers for Node.js — node.testcontainers.org
3. リポジトリ内: `apps/web/test/project-access.test.ts` , `apps/web/test/load-project-view.test.ts`
4. リポジトリ内: `.github/workflows/ci.yml`

VECTA 進捗レポート — 2026-07-26